

Exercise: Hinge-Flip Motion in a Closed Rigid Chain

1 Exercise statement

Consider a closed planar chain (a ring) consisting of N rigid bars of equal length D . The vertices are

$$p_i \in \mathbb{R}^2, \quad i \in \mathbb{Z}_N,$$

and adjacent vertices satisfy the rigid constraints

$$\|p_{i+1} - p_i\| = D, \quad \forall i \in \mathbb{Z}_N.$$

Assume the initial configuration is a regular N -gon approximating a circle of circumference $C = 369$, i.e.

$$R = \frac{C}{2\pi}, \quad p_i^0 = R(\cos(2\pi i/N), \sin(2\pi i/N)),$$

and bar length equals the chord length

$$D = 2R \sin\left(\frac{\pi}{N}\right).$$

Task

Pick a vertex index j . We want to realize a motion where the vertex p_j ends up on the *opposite branch* of the local hinge formed by the triple

$$(p_{j-1}, p_j, p_{j+1}),$$

meaning the triangle (p_{j-1}, p_j, p_{j+1}) changes orientation (the “hinge angle sign flips”) while $\|p_{i+1} - p_i\| = D$ remains satisfied.

Correct qualitative behavior to reproduce

A continuous motion that changes the local orientation is only possible if the two neighbors p_{j-1} and p_{j+1} move outward first, increasing the chord length

$$L = \|p_{j+1} - p_{j-1}\|$$

up to the singular value $L = 2D$ (collinearity), at which point the two hinge branches coalesce and the orientation can switch. Afterwards the chord may be reduced again while staying on the flipped branch.

What to verify

At each discrete step of the motion, compute all edge lengths $\|p_{i+1} - p_i\|$ and report the deviations from the nominal bar length D . The maximal deviation should remain near numerical tolerance.

2 Derivation of the hinge-flip solution

2.1 Local hinge geometry (two-circle intersection)

Fix two endpoints

$$A = p_{j-1}, \quad C = p_{j+1}.$$

The middle vertex $B = p_j$ must satisfy

$$\|B - A\| = D, \quad \|B - C\| = D.$$

Thus B lies on the intersection of two circles of radius D centered at A and C .

Let

$$c = C - A, \quad L = \|c\|, \quad m = \frac{A + C}{2}.$$

Define a unit normal to the chord AC :

$$n = \frac{1}{L} (-c_y, c_x).$$

Then the two intersection points are

$$B = m \pm h n, \quad h = \sqrt{D^2 - \left(\frac{L}{2}\right)^2}.$$

The $\pm$ choices correspond to the two orientation branches of the triangle ABC .

2.2 Why the neighbors must move

If A and C are held fixed, B has only two feasible positions (discrete). To move continuously from the “plus” to “minus” position, these must merge, which requires

$$h = 0 \iff \frac{L}{2} = D \iff \boxed{L = 2D}.$$

At $L = 2D$ the triangle is degenerate and A, B, C become collinear. Therefore a branch switch requires changing L , which in the closed chain requires coordinated motion of (at least) neighboring vertices.

2.3 A discrete continuation path (open–flip–close)

Define the motion in three stages:

1. **Open:** Increase $L = \|p_{j+1} - p_{j-1}\|$ from its initial value L_0 toward $2D$ by moving A and C outward along their chord direction.
2. **Flip:** At near-singularity $L \approx 2D$, switch the branch sign ($+ \rightarrow -$ or $- \rightarrow +$). At exactly $L = 2D$, both branches coincide, so positions are continuous.
3. **Close:** Reduce L back to L_0 while staying on the flipped branch.

2.4 Global constraint enforcement via projection

The open/close driver changes only p_{j-1}, p_j, p_{j+1} . To keep the entire ring consistent, enforce all constraints by iteratively projecting each edge $(i, i + 1)$ back to length D . This yields a practical toy model:

$$(\text{driver step}) \Rightarrow (\text{local hinge set}) \Rightarrow (\text{global projection to } \|p_{i+1} - p_i\| = D).$$

For correctness checks we compute edge length deviations

$$\delta_i = \|p_{i+1} - p_i\| - D.$$

3 Implementation

3.1 Bun/TypeScript: generate JSON for Plotly

The following program generates a file `chain_flip.json` containing:

- `frames[k].x[i]`, `frames[k].y[i]`: vertex coordinates at step k ,
- `frames[k].edgeLen[i]`: $\|p_{i+1} - p_i\|$,
- `frames[k].edgeDiff[i]`: deviation $\|p_{i+1} - p_i\| - D$,
- `frames[k].maxAbsDiff`: maximum absolute deviation at step k .

Listing 1: `make_chain_flip.ts` (Bun). Produces `chain_flip.json`.

```
#!/usr/bin/env bun

type Vec2 = [number, number];

function add(a: Vec2, b: Vec2): Vec2 { return [a[0]+b[0], a[1]+b[1]]; }
function sub(a: Vec2, b: Vec2): Vec2 { return [a[0]-b[0], a[1]-b[1]]; }
function scale(a: Vec2, s: number): Vec2 { return [a[0]*s, a[1]*s]; }
function norm(a: Vec2): number { return Math.hypot(a[0], a[1]); }
function perp(a: Vec2): Vec2 { return [-a[1], a[0]]; }
function unit(a: Vec2): Vec2 {
  const n = norm(a);
  return n < 1e-14 ? [0,0] : [a[0]/n, a[1]/n];
}

function projectEdge(p: Vec2[], i: number, i1: number, D: number, pinned: Set<number>) {
  const d = sub(p[i1], p[i]);
  const len = norm(d);
  if (len < 1e-14) return;
  const err = len - D;
  if (Math.abs(err) < 1e-12) return;

  const n = scale(d, 1/len);

  if (pinned.has(i) && pinned.has(i1)) return;
  if (pinned.has(i)) {
    p[i1][0] -= err*n[0]; p[i1][1] -= err*n[1];
  } else if (pinned.has(i1)) {
    p[i][0] += err*n[0]; p[i][1] += err*n[1];
  } else {
    p[i][0] += (err/2)*n[0]; p[i][1] += (err/2)*n[1];
    p[i1][0] -= (err/2)*n[0]; p[i1][1] -= (err/2)*n[1];
  }
}

function projectAllEdges(p: Vec2[], D: number, pinned: Set<number>, sweeps = 120) {
  const N = p.length;
  for (let it = 0; it < sweeps; it++) {
    let maxErr = 0;
    for (let i = 0; i < N; i++) {
      const i1 = (i+1) % N;
      const len = norm(sub(p[i1], p[i]));
      maxErr = Math.max(maxErr, Math.abs(len-D));
      projectEdge(p, i, i1, D, pinned);
    }
    if (maxErr < 1e-10) break;
  }
}
```

```

    }
}

function hingeCandidates(A: Vec2, C: Vec2, D: number) {
    const c = sub(C, A);
    const L = norm(c);
    const m = scale(add(A, C), 0.5);

    const half = L/2;
    const inside = Math.max(0, D*D - half*half);
    const h = Math.sqrt(inside);

    const n = L < 1e-14 ? ([0,0] as Vec2) : scale(perp(c), 1/L);
    const plus = add(m, scale(n, +h));
    const minus = add(m, scale(n, -h));
    return { plus, minus };
}

function chooseSigmaClosest(A: Vec2, C: Vec2, Bcur: Vec2, D: number): number {
    const { plus, minus } = hingeCandidates(A, C, D);
    const dPlus = norm(sub(plus, Bcur));
    const dMin = norm(sub(minus, Bcur));
    return dPlus <= dMin ? +1 : -1;
}

function hingeB(A: Vec2, C: Vec2, D: number, sigma: number): Vec2 {
    const { plus, minus } = hingeCandidates(A, C, D);
    return sigma === +1 ? plus : minus;
}

function edgeStats(p: Vec2[], D: number) {
    const N = p.length;
    const edgeLen = Array(N).fill(0);
    const edgeDiff = Array(N).fill(0);
    let maxAbsDiff = 0;

    for (let i=0; i<N; i++) {
        const i1 = (i+1)%N;
        const l = norm(sub(p[i1], p[i]));
        const d = l - D;
        edgeLen[i]=l;
        edgeDiff[i]=d;
        maxAbsDiff = Math.max(maxAbsDiff, Math.abs(d));
    }
    return { edgeLen, edgeDiff, maxAbsDiff };
}

function makeFrames(N: number, C: number, j: number, stepsOpen=60, stepsClose=60) {
    const R = C/(2*Math.PI);
    const alpha = (i: number) => (2*Math.PI*i)/N;

    const p0: Vec2[] = Array.from({length:N}, (_,i)=>[
        R*Math.cos(alpha(i)),
        R*Math.sin(alpha(i)),
    ]);

    const D = 2*R*Math.sin(Math.PI/N);

    const jm = (j-1+N)%N;
    const jp = (j+1)%N;

```

```

const A0 = p0[jm];
const C0 = p0[jp];
const c0 = sub(C0, A0);
const L0 = norm(c0);
const m0 = scale(add(A0, C0), 0.5);
const chat = unit(c0);

const Lsing = 2*D;
// approach singularity closely but not exactly (numeric stability)
const eps = 1e-3 * D;
const Lopen = Math.min(Lsing - eps, Lsing - eps);

let p: Vec2[] = p0.map(v=>[v[0], v[1]]);

const sigma0 = chooseSigmaClosest(p[jm], p[jp], p[j], D);
const sigma1 = -sigma0;

function setChord(L: number) {
  const half = L/2;
  const A = sub(m0, scale(chat, half));
  const Cc = add(m0, scale(chat, half));
  p[jm]=[A[0],A[1]];
  p[jp]=[Cc[0],Cc[1]];
}

function enforce(sigma: number) {
  const B = hingeB(p[jm], p[jp], D, sigma);
  p[j]=[B[0],B[1]];
  const pinned = new Set<number>([jm, j, jp]);
  projectAllEdges(p, D, pinned, 160);
  p[j]=[B[0],B[1]];
}

const frames: any[] = [];
frames.push({ x: p.map(q=>q[0]), y: p.map(q=>q[1]), ...edgeStats(p, D) });

// open
for (let s=1; s<=stepsOpen; s++) {
  const t = s/stepsOpen;
  const L = L0 + (Lopen-L0)*t;
  setChord(L);
  enforce(sigma0);
  frames.push({ x: p.map(q=>q[0]), y: p.map(q=>q[1]), ...edgeStats(p, D) });
}

// close with flipped branch
for (let s=1; s<=stepsClose; s++) {
  const t = s/stepsClose;
  const L = Lopen + (L0-Lopen)*t;
  setChord(L);
  enforce(sigma1);
  frames.push({ x: p.map(q=>q[0]), y: p.map(q=>q[1]), ...edgeStats(p, D) });
}

return {
  meta: { N, C, R, D, j, jm, jp, sigma0, sigma1, stepsOpen, stepsClose },
  p0: { x: p0.map(q=>q[0]), y: p0.map(q=>q[1]) },
  frames
};

```

```

}

// ---- CLI ----
const N = parseInt(process.argv[2] ?? "12", 10);
const j = parseInt(process.argv[3] ?? "0", 10);
const out = process.argv[4] ?? "chain_flip.json";

const data = makeFrames(N, 369, ((j%N)+N)%N);
await Bun.write(out, JSON.stringify(data, null, 2));
console.log("wrote", out);

```

Run

```
bun run make_chain_flip.ts 12 0 chain_flip.json
```

3.2 HTML + Plotly: visualize and verify distances at each step

This HTML reads `chain_flip.json`, shows:

- the ring configuration for the current step,
- the per-edge deviations $\delta_i = \|p_{i+1} - p_i\| - D$,
- the maximum deviation over time.

Listing 2: `check.html`. Open in a browser (serve locally for fetch).

```

<!doctype html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Chain Flip Check</title>
  <script src="https://cdn.plot.ly/plotly-2.30.0.min.js"></script>
  <style>
    body { font-family: system-ui; margin: 16px; }
    #row { display: flex; gap: 16px; }
    #plotShape, #plotErr { width: 48vw; height: 70vh; }
    pre { background: #f6f6f6; padding: 10px; border-radius: 6px; }
    input[type=range] { width: 60vw; }
  </style>
</head>
<body>
  <h2>Rigid ring hinge-flip: step-by-step correctness check</h2>
  <div>
    <input id="step" type="range" min="0" max="0" value="0" step="1" />
    <span id="stepLabel"></span>
  </div>
  <div id="row">
    <div id="plotShape"></div>
    <div id="plotErr"></div>
  </div>
  <pre id="info"></pre>

  <script>
  async function main() {
    const data = await (await fetch('./chain_flip.json')).json();
    const { meta, p0, frames } = data;

    const step = document.getElementById('step');
    const stepLabel = document.getElementById('stepLabel');
  }
  </script>

```

```

const info = document.getElementById('info');

step.max = String(frames.length - 1);

// Precompute maxAbsDiff series
const t = frames.map((_,k)=>k);
const maxAbs = frames.map(f => f.maxAbsDiff);

// Plot: max error over time
Plotly.newPlot('plotErr', [{
  x: t, y: maxAbs, mode: 'lines+markers', name: 'max |len-D|'
}], {
  title: 'Constraint deviation over steps',
  xaxis: { title: 'step' },
  yaxis: { title: 'max |len-D|' }
});

function edgeLensFromXY(x, y) {
  const N = x.length;
  const len = new Array(N).fill(0);
  for (let i=0;i<N;i++) {
    const i1=(i+1)%N;
    const dx=x[i1]-x[i], dy=y[i1]-y[i];
    len[i]=Math.hypot(dx,dy);
  }
  return len;
}

function update(k) {
  const f = frames[k];

  // Compute lengths again in JS to confirm the JSON values
  const lensJS = edgeLensFromXY(f.x, f.y);
  const diffsJS = lensJS.map(L => L - meta.D);
  const maxAbsJS = Math.max(...diffsJS.map(d=>Math.abs(d)));

  // Shape plot (polygon)
  const xClosed = [...f.x, f.x[0]];
  const yClosed = [...f.y, f.y[0]];

  Plotly.react('plotShape', [
    { x: [...p0.x, p0.x[0]], y: [...p0.y, p0.y[0]],
      mode: 'lines', name: 'initial', line: { width: 1, color: 'rgba(120,120,120,0.6)'} },
    { x: xClosed, y: yClosed,
      mode: 'lines+markers', name: 'current', marker: { size: 7 } }
  ], {
    title: `Shape at step ${k}`,
    xaxis: { scaleanchor: 'y' },
    yaxis: { },
    showlegend: true
  });

  // Update error plot marker
  Plotly.restyle('plotErr', {
    'marker.size': [t.map(i => i === k ? 10 : 4)]
  }, [0]);

  // Text info + per-edge diffs table
  const lines = [];

```

```

lines.push(`meta: N=${meta.N}, D=${meta.D.toFixed(8)}, j=${meta.j}, jm=${meta.jm}, jp=
  ${meta.jp}`);
lines.push(`step ${k}/${frames.length-1}`);
lines.push(`maxAbsDiff (JSON) = ${f.maxAbsDiff.toExponential(3)}`);
lines.push(`maxAbsDiff (JS recompute) = ${maxAbsJS.toExponential(3)}`);
lines.push('');
lines.push(`edge i->i+1 : len(JSON), diff(JSON) | len(JS), diff(JS)`);
for (let i=0;i<meta.N;i++) {
  const i1=(i+1)%meta.N;
  const lJ = f.edgeLen[i], dJ = f.edgeDiff[i];
  const lS = lensJS[i], dS = diffsJS[i];
  lines.push(
    `${String(i).padStart(2,'0')}->${String(i1).padStart(2,'0')}: ` +
    `${lJ.toFixed(10)} ${dJ.toExponential(2)} | ` +
    `${lS.toFixed(10)} ${dS.toExponential(2)}`
  );
}
info.textContent = lines.join('\n');

stepLabel.textContent = `u = ${(k/(frames.length-1)).toFixed(3)}`;
}

step.addEventListener('input', () => update(parseInt(step.value, 10)));
update(0);
}

main().catch(e => {
  document.body.innerHTML += `

```
>'+String(e)+'</pre>`;
});
</script>
</body>
</html>

```


```

Serve locally Because browsers block `fetch` from `file://`, run a tiny local server:

```

# in the directory containing check.html and chain_flip.json
python3 -m http.server 8000
# then open:
# http://localhost:8000/check.html

```

4 What a correct result looks like

A correct implementation shows:

- During intermediate steps (around the singular phase), the neighbors p_{j-1} and p_{j+1} visibly move outward.
- The middle vertex p_j crosses to the opposite hinge branch after the open-flip-close sequence.
- All edge-length deviations δ_i remain near numerical tolerance (e.g. $\max_i |\delta_i| \lesssim 10^{-8}$ to 10^{-6} depending on iteration counts and floating-point error).